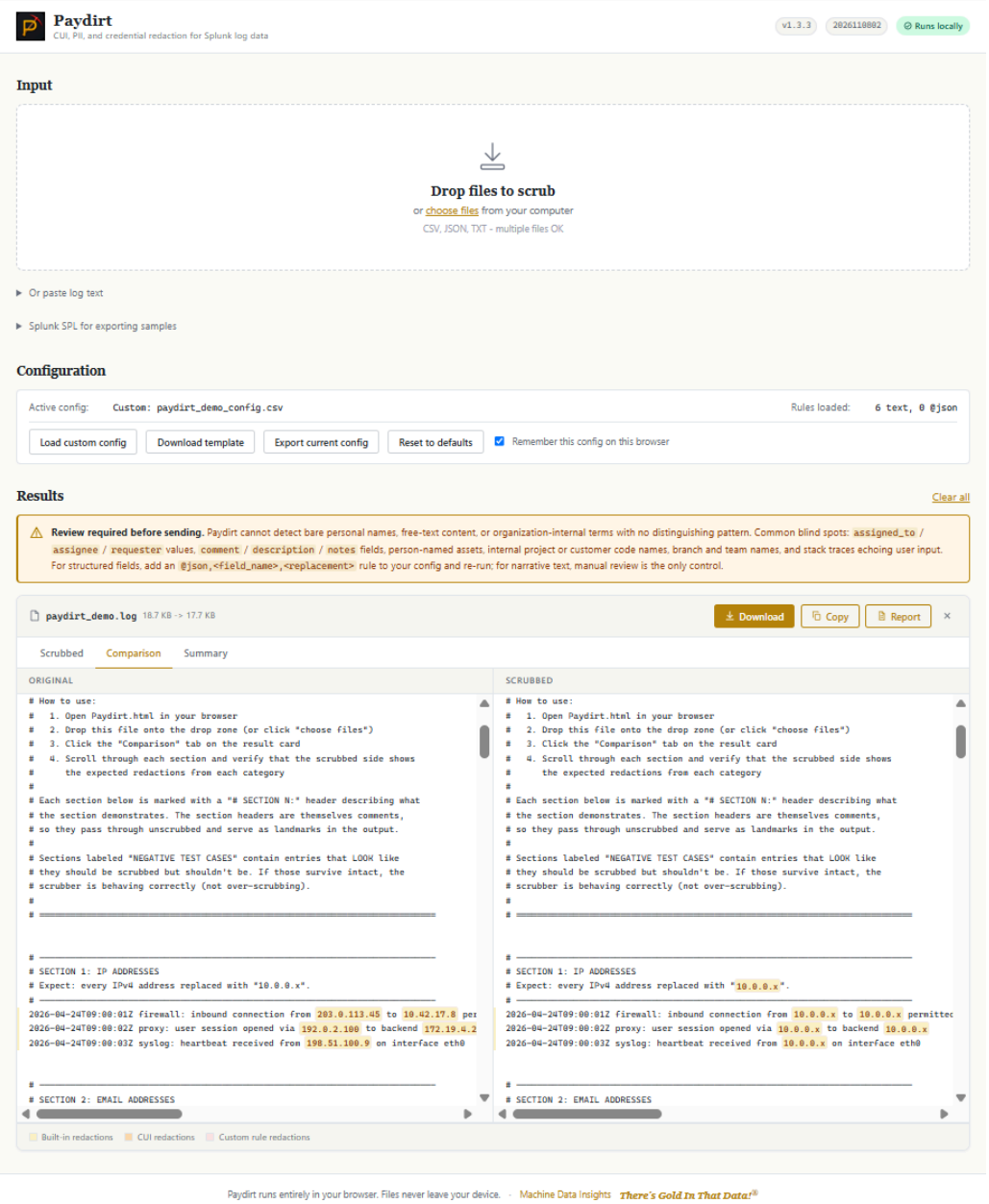
Paydirt + Log Scrubber

Paydirt is a self-contained browser tool plus matching Python CLI for scrubbing sensitive data from Splunk® or other log data exports so they can be mined for value. Drop a file, get a sanitized version - no install, no network calls, nothing leaves your machine. CMMC, HIPAA, and GDPR aware. Environment customizable.

Paydirt was created and is used by Machine Data Insights as part of the CIM Normalization Pipeline - see the info page on the Machine Data Insights website. *There's Gold In That Data!*®

⬇ Download Paydirt v1.3.3 - browser tool, ~290 KB, runs offline. Save the file, double-click to open. All files below ↓



Paydirt's Comparison view shows original and scrubbed output side by side, with per-field highlighting that tells you exactly what changed.

What Gets Redacted

Out of the box, with zero configuration, Paydirt finds and replaces:

Network & identity - IPv4/IPv6 addresses, AWS `ip-` hostnames, email addresses, FQDN hostnames, UNC paths (`\\server\share`), domain usernames (`CORP\jsmith`), MAC addresses

Credentials & tokens - PEM private key blocks, AWS access keys (`AKIA...`, `ASIA...`), GitHub PATs (`ghp_`, `gho_`, ...), Slack tokens (`xoxb-`, `xoxp-`, ...), Stripe keys (`sk_live_`, `sk_test_`), JWTs, Google API keys (`AIza...`), HTTP `Authorization` header values, URL query-string credentials (`?password=`, `&api_key=`, ...)

PII / PHI - SSNs (valid issuance ranges only), credit card numbers (Luhn-validated), NPIs (validated per 45 CFR 162.406), formatted US phone numbers, Windows user SIDs (well-known system SIDs like `S-1-5-18` preserved)

CUI markings (CMMC / NIST SP 800-171) - CUI banner markings (`CUI`, `CUI//SP-PRVCY`, `CUI//SP-PROPIN`, `CUI//SP-EXPT`, `CUI//SP-LEI`, `CUI//SP-CTI`, `CONTROLLED//...`), portion markings (`(CUI)`, `(U//FOUO)`, `(U//SBU)`, `(U//LES)`, `(C)`, `(U)`), legacy markings (FOUO, SBU, LES, OUO, LIMDIS, NOFORN, FEDCON, ORCON), and CUI-adjacent flags (ITAR, EAR99, ECCN, DD 254, FCI) - full-value redaction with metadata-only placeholder

Plus whatever else you tell it to - text substitution, JSON field targeting at any nesting depth (including dotted paths like `userIdentity.arn`), AWS/Azure/GCP tag structures, and random replacement pools via a simple CSV config

Validators run inside the matchers, so ordinary 10-to-19-digit numbers (order IDs, tracking numbers, timestamps) aren't mistaken for SSNs, credit cards, or NPIs. See Built-in Scrubbing below for exact patterns and replacement values.

Downloads

Latest release: v1.3.3

The tool itself:

- Paydirt.html - browser tool. Save, double-click to open, drop log files on it. No install required.
- log_scrubber.py - Python CLI. Requires Python 3.9+. No other dependencies.

Configuration:

- log_scrubbing_config.csv - default config used by both tools (optional - both have built-in defaults).

Documentation: printable references for end users, compliance reviewers, and security teams evaluating Paydirt for regulated environments.

- LOG_SCRUBBER_GUIDE.pdf - CLI quick-start guide, printable.
- COMPLIANCE_PRIMER.pdf - plain-language background on the CMMC, HIPAA, and GDPR obligations Paydirt is designed to support.
- SECURITY_ARCHITECTURE.pdf - architecture and security review for compliance reviewers, covering data flow, trust boundaries, threat model, and code provenance.

Demo files (for trying Paydirt or evaluating it for compliance use):

- paydirt_demo.log - example log file exercising every scrubbing category.
- paydirt_demo_config.csv - companion config that lights up the custom-rule section of the demo.

What It Does

Log scrubbing tools for removing CUI, PII, PHI, credentials, and other sensitive data from Splunk field-value exports and log samples. Two interchangeable implementations of the same scrubbing algorithm:

- `Paydirt.html` - a self-contained browser tool. Download the file, double-click, scrub logs in the browser. No install, no network calls, no data leaves your machine. Good for environments where you can't install Python (locked-down corporate laptops, air-gapped networks).
- `log_scrubber.py` - a Python CLI and importable library. Good for automation, batch processing, and integration into existing pipelines.

Both tools use the same configuration format (`log_scrubbing_config.csv`) and produce identical output for identical input (verified by `shared-tests/run_parity.py`).

Compliance coverage: CMMC (CUI marking detection per NIST SP 800-171), HIPAA (PHI identifiers including SSN, Luhn-validated payment cards, NPI, phone), GDPR (personal data including IPs and device identifiers).

No dependencies beyond Python 3.9+ standard library for the CLI, and no dependencies at all for the browser tool (pure HTML/CSS/JS, runs offline).

Editing the Scrub Config

Both tools read their rules from `log_scrubbing_config.csv` - a plain CSV you can edit in any text editor or spreadsheet. Each non-comment line is one rule; lines starting with `#` are ignored. This is a quick primer covering the everyday cases and one gotcha; the full reference is in Configuration further down.

Text replacements

The simplest rule replaces a literal string wherever it appears. The format is `search_term,mode,replacement`. Use `single` mode when the replacement is always the same:

```
# search_term,single,replacement
ACME Corp,single,CompanyA
```

Use `random` mode to vary the replacement - each occurrence independently picks one value from the list. Any value that itself contains commas must be quoted:

```
john.doe@example.com,random,"user1@company.com,user2@company.com,user3@company.com
"
```

JSON field replacements

Prefix a rule with `@json` to replace the *value* of a named field in JSON data, whatever that value happens to be. The simplest form is three columns (`@json,fieldName,replacement`):

```
@json,sourceIPAddress,10.0.0.x
```

For nested data you can use dot notation to target a field only under a specific parent, and `random` mode works here too:

```
# Match 'arn' only when nested under 'userIdentity' (not any other arn)
@json,userIdentity.arn,arn:aws:iam::000000000000:user/REDACTED

# Vary the value across occurrences
@json,accountId,random,"000000000001,000000000002,000000000003"
```

Rule order matters: specific before broad

Text rules run **top to bottom**, and each rule is applied to the output of the rules above it. A broad rule earlier in the file can therefore "eat" the term a more specific rule below it was meant to catch - by the time the specific rule runs, its search term has already been rewritten and no longer exists in the text, so the specific rule silently does nothing.

Broken - the broad `ACME` rule runs first and turns `ACME Corp` into `COMP Corp`, so the `ACME Corp` rule never matches:

```
ACME,single,COMP
ACME Corp,single,CompanyA      # never fires - "ACME Corp" is already gone
```

Fixed - list the more specific rule first, so it claims its text before the broad rule sweeps up whatever remains:

```
ACME Corp,single,CompanyA      # matches first
ACME,single,COMP               # then mops up any remaining bare "ACME"
```

The rule: whenever two text rules could match the same substring, order them **most specific → most general**. (`@json` field rules are keyed by field name rather than by raw substring, so they don't collide this way - this ordering concern applies to text-substitution rules.)

Documentation

- This README is the canonical project documentation
- `docs/LOG_SCRUBBER_GUIDE.md` - CLI quick-start, condensed for users who just want to scrub their first file
- `docs/COMPLIANCE_PRIMER.md` - plain-language compliance background
- `docs/SECURITY_ARCHITECTURE.pdf` - architecture and security review for compliance reviewers, covering data flow, trust boundaries, threat model, and code provenance
- `paydirt/README.md` - browser tool source layout and build process

Try It Yourself

The demo files (`paydirt_demo.log` and `paydirt_demo_config.csv`) are also included at the project root if you'd rather work from a cloned copy of the repo. To exercise the demo from the command line:

```
# Built-in patterns only:
python log_scrubber.py paydirt_demo.log

# Built-in patterns plus the companion config (lights up Section 25):
python log_scrubber.py paydirt_demo.log --config paydirt_demo_config.csv
```

Each section of the demo file is marked with a `# SECTION N:` header describing what it demonstrates. Reviewers evaluating the tool for compliance purposes can inspect each section's expected behavior, run the scrubber, and visually confirm that every category of sensitive data gets correctly redacted (and that negative test cases remain untouched).

Working With Your Own Splunk Data

To pull data from your own Splunk environment for scrubbing, Paydirt includes ready-to-run SPL examples directly in the UI. Click **Splunk SPL for exporting samples** in the Input section to expand a panel with three searches:

- **Discover sourcetypes** (last 30 days) - uses `tstats` against the metadata index to list sourcetypes with their indexes, sources, and event counts. The example uses `index=* AND sourcetype=*` to list everything as a starting point - narrow the `index=...` and `sourcetype=...` filters to match your environment, and keep them specific in large deployments or the search can run for a very long time. The 30-day window also avoids surfacing retired sourcetypes. Run this first to pick a sourcetype for the other two searches; the export also doubles as the input for the Data Refinery *Sourcetype CSV* entry.
- **Field-value samples** (last 7 days) - uses `fieldsummary` to extract the distinct values present in each field across a sourcetype, with the housekeeping fields filtered out. This is the right input for understanding the shape of your data and building CIM normalization rules.
- **Log samples** (last 1 day) - uses `dedup punct | head 30` to grab a representative set of distinct event patterns. This is the right input for working with full event text, including any embedded JSON. For a more complete picture you can drop the `| head 30` cap (keeping the `dedup punct`) so every distinct pattern is captured - useful when rare events carry important data that the 30-event cap might exclude, as long as the result set stays a manageable size.

Each block has a **Copy** button to drop the SPL into your clipboard with one click, and a **Save as** hint suggesting a filename convention (`<company>_<environment>_<sourcetype>_...csv`) so exports stay organized across environments. The **Field-value samples** search has a `{sourcetype}` placeholder to fill in; the **Log samples** search uses `index=* sourcetype=* source=*` and runs as-is - narrow those wildcards if your environment is large. Run the search in Splunk Web, click **Export** → **CSV**, and drop the saved file onto Paydirt to scrub it.

> **Don't scrub the discovery export.** Sourcetype, index, and source names are structural metadata, not log content - they need to stay verbatim so downstream consumers can use them. Index names, for example, often feed CIM macros where any redaction will break the macro, and sourcetype names have

> to match exactly when used in subsequent searches. Only scrub the discovery export if a name itself obviously contains sensitive data (e.g., a customer name baked into an index name); otherwise pass it through to the Data Refinery *Sourcetype CSV* entry as-is. The field-value and log-sample exports, by contrast, *should* be scrubbed before sharing.

The field-value and log-sample searches work with `log_scrubber.py` too - export from Splunk, then run `python log_scrubber.py your_export.csv`.

Audit Report

For audit and accountability workflows, each result in the browser tool has a **Report** button next to **Download**. It produces a retainable record of *what* scrubbing was performed:

- tool version and build number, plus generation timestamps (local and UTC)
- source and scrubbed file names with their UTF-8 byte sizes
- **SHA-256 hashes** of the original and scrubbed text, so the report can be cryptographically tied to the exact files and any later tampering is evident
- the config source and its rule counts
- per-category and per-custom-rule redaction counts

The report **contains no sensitive data from the source file by design** - it proves scope and volume without disclosing the values that were removed, and is generated entirely in the browser (SubtleCrypto for the hashes). The file is a human-readable header followed by a machine-readable JSON block.

The redaction counts use the same heuristic as the Summary tab and are documented in the report as a summary, **not** a line-level certification; random-mode rules are non-deterministic. The audit report is browser-only in this release; a `log_scrubber.py` counterpart is planned.

Repository Layout

```
paydirt/
├── README.md                   this file
├── Paydirt.html                the browser tool (single-file distribution)
├── log_scrubber.py             the Python CLI (single-file distribution)
├── log_scrubbing_config.csv    default config (shared by both tools)
│
├── paydirt_demo.log            demo log file for reviewers
├── paydirt_demo_config.csv     companion demo config
│
├── docs/                       compliance and architecture documentation
│   ├── COMPLIANCE_PRIMER.md    plain-language compliance background
│   ├── LOG_SCRUBBER_GUIDE.md   CLI quick-start guide
│   └── SECURITY_ARCHITECTURE.pdf architecture and security review document
│
├── cli/                        Python CLI tests
│   ├── README.md
│   └── test_scrubber.py
│
├── paydirt/                    browser tool source (build artifacts land
│   ├── README.md               at the repo root, not here)
```

```
│   ├── build.py
│   ├── src/
│   │   ├── index.html, styles.css, scrubber.js, app.js
│   └── tests/
│       ├── test-node.js         JS unit tests
│       └── smoke-test.py        headless browser smoke test
│
└── shared-tests/                parity tests between Python and JS
    ├── README.md
    ├── run_parity.py
    └── input/                   fixtures fed through both implementations
```

Quick Start - CLI

The remainder of this README covers the Python CLI. For the browser tool, see `paydirt/README.md` (for developers) or just open `Paydirt.html` in your browser (for users).

```
# Auto-detect mode from file contents (easiest)
python log_scrubber.py my_fields.csv

# Batch multiple files - mode auto-detected per file
python log_scrubber.py ~/Downloads/*.csv

# Explicit mode (v1 syntax, still supported)
python log_scrubber.py fieldsummary my_fields.csv
python log_scrubber.py samples my_events.csv

# Include raw values for comparison
python log_scrubber.py fieldsummary my_fields.csv --include-raw

# Disable CUI marking detection (if your data isn't CUI-regulated)
python log_scrubber.py my_events.csv --no-cui
```

Output is written to `my_fields_scrubbed_20260420_103045.csv` (auto-timestamped).

Output Format

Fieldsummary Mode

By default, the output contains **scrubbed values only** - the raw values column is removed so the output is safe to share:

```
Field Name, Scrubbed Values, Count, Distinct Count
```

Use `--include-raw` to keep the original raw values alongside the scrubbed values (useful for reviewing what was changed):

```
Field Name, Raw Values, Scrubbed Values, Count, Distinct Count
```

Samples Mode

Log sample events are scrubbed in place - the output file contains only scrubbed events.

Step-by-Step Workflow

1. Discover Sourcetypes (optional)

If you don't already know which index and sourcetype you want, run this first against your Splunk metadata index:

```
| tstats count where index=* AND sourcetype=* earliest=-30d BY sourcetype, index,
source
| stats values(index) as indexes, values(source) as sources, sum(count) as
event_count by sourcetype
| sort sourcetype
```

As written it lists every sourcetype (`sourcetype=*`); narrow it to a specific sourcetype or wildcarded filter (e.g., `cisco:asa` or `cisco:*`) and adjust the `index=...` filter for your own environment. Keep the filter specific in large deployments or the search can run for a very long time. The `earliest=-30d` window also avoids surfacing sourcetypes that no longer exist.

Click **Export** → choose **CSV** → save the file using the convention `<company>_<environment>_sourcetype_discovery.csv`.

This lists each matching sourcetype with its indexes, sources, and event counts so you can pick a target for the next two searches. **Do not scrub this export** - sourcetype and index names need to stay accurate for downstream use (CIM macros, Data Refinery *Sourcetype CSV* entry, etc.). Only redact a name by hand if it itself contains sensitive content.

2. Export Field Values from Splunk Web

Run this SPL in Splunk Web (adjust index, sourcetype, and time range):

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-7d@d latest=now
| fieldsummary maxvals=15
| search field!="_*" AND field!="date_*" AND field!="linecount"
  AND field!="punct" AND field!="timestartpos" AND field!="timeendpos"
  AND field!="splunk_server_group"
```

Click **Export** → choose **CSV** → save the file using the convention `<company>_<environment>_<sourcetype>_fieldsummary.csv`.

3. Export Log Samples from Splunk Web

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-1d@d latest=now
| dedup punct | head 30
```

Click **Export** → choose **CSV** → save the file using the convention `<company>_<environment>_<sourcetype>_log-samples.csv`.

Tip: consider dropping the `| head 30` cap (but keeping `| dedup punct`) to capture *every* distinct event pattern. `dedup punct` already collapses the result set to one event per unique punctuation shape, so the unbounded query usually returns only a few dozen to a few hundred rows - completely manageable for most sourcetypes. The 30-event cap can silently hide infrequent-but-important patterns (rare error variants, privileged-user actions, edge-case payloads), so for small or medium sourcetypes the unbounded form is often the better choice. Keep the cap for very high-cardinality sourcetypes where the unbounded result would be too large.

Alternatively, you can copy/paste raw events into a plain `.txt` file (one event per line). The scrubber auto-detects the format.

4. Scrub the Exports

Skip the discovery export from Step 1 - it should pass through untouched. Run the scrubber on the field-value and log-sample exports. With auto-detection (recommended), you don't need to specify the mode:

```
# Scrub one file
python log_scrubber.py guardduty_fields.csv

# Scrub everything from today's Downloads
python log_scrubber.py ~/Downloads/*.csv
```

The scrubber inspects each file's header and dispatches to the correct mode. You can still use explicit mode keywords if you prefer:

```
python log_scrubber.py fieldsummary guardduty_fields.csv
python log_scrubber.py samples guardduty_samples.csv
```

5. Review and Send

Check the `*_scrubbed_*` output files to verify sensitive data was replaced, then email the scrubbed files for processing.

> ***Read every line before you send.*** *Paydirt redacts what it can recognize by shape (IPs, emails, tokens, SSNs, CUI markings) or by structured field name. It* ***cannot*** *detect bare personal names, free-text content, or organization-internal terms that have no distinguishing pattern. Common blind spots: an* `assigned_to` */* `assignee` */* `requester` */* `owner` *value like* `John Smith` *sitting in an unstructured* `_raw`

> *event, a `comment` / `description` / `notes` field containing customer names or case details, a person-named asset like `JSMITH-LAPTOP`, internal project or customer code names, branch and team names, and stack traces that echo user input. If a sensitive value lives in its own JSON key, CSV column, or fieldsummary field, you can target it with an `@json,<field_name>,<replacement>` rule and re-run - the `@json` shortcut covers all three. If it's buried in narrative text, no automated tool can find it reliably. Your eyes are the control of last resort.*

Configuration

Config File Location

The scrubber automatically looks for `log_scrubbing_config.csv` in these locations (first match wins):

1. Current working directory: `./log_scrubbing_config.csv`
2. Data subdirectory: `./data/log_scrubbing_config.csv`
3. Same directory as the script: `<script_dir>/log_scrubbing_config.csv`
4. Data subdirectory of script: `<script_dir>/data/log_scrubbing_config.csv`

The simplest approach is to **place the config file in the same folder as `log_scrubber.py`**.

You can also specify a config explicitly with `--config`:

```
python log_scrubber.py fieldsummary fields.csv --config /path/to/my_rules.csv
```

Built-in Scrubbing (Always Active)

Even without a config file, the scrubber applies these patterns automatically. They can be disabled as a group with `--no-builtin`.

Network & identity (v1.0):

Pattern	Replacement
IP addresses (`192.168.1.50`)	`10.0.0.x`
AWS ip- hostnames (`ip-10-50-26-117`)	`ip-10-0-0-x`
Email addresses (`admin@corp.com`)	`user@example.com`
FQDN hostnames (`server01.company.com`)	`host.example.com`
UNC paths (`\\server\share`)	`\\SERVER\SHARE`
Domain usernames (`CORP\jsmith`)	`DOMAIN\user`
MAC addresses (`00:1A:2B:3C:4D:5E`)	`00:00:00:00:00:00`

Credentials & tokens (v1.1):

Pattern	Replacement
PEM private key blocks	`[REDACTED_PRIVATE_KEY]`
AWS access keys (`AKIA...`, `ASIA...`)	`AKIAREDACTEDREDACTED`
GitHub PATs (`ghp_...`, `gho_...`, etc.)	`ghp_REDACTED`
Slack tokens (`xoxb-...`, `xoxp-...`, etc.)	`xoxb-REDACTED`
Stripe keys (`sk_live_...`, `sk_test_...`)	`sk_test_REDACTED`
JWTs (`eyJ...` 3-segment)	`eyJREDACTED.REDACTED.REDACTED`
Google API keys (`AIza...`)	`AIzaREDACTED`
HTTP `Authorization` header values	`Authorization: Bearer REDACTED`
URL query credentials (`?password=`, `&api_key=`, etc.)	`...=REDACTED`

PII / PHI (v1.1):

Pattern	Replacement
SSN (formatted `XXX-XX-XXXX`, valid ranges only)	`000-00-0000`
Credit card numbers (Luhn-validated)	`4111-1111-1111-1111`
NPI (10-digit, validated per 45 CFR 162.406)	`1234567893`
Phone numbers (formatted, US)	`555-555-5555`
Windows SIDs (user SIDs only, `S-1-5-21-...`)	`S-1-5-21-0-0-0-0`

Luhn and NPI validators run inside the match, so ordinary 10-to-19-digit numbers (order IDs, tracking numbers, timestamps) are left alone. Well-known system SIDs like `S-1-5-18` (SYSTEM) are preserved.

CUI markings (v1.1): see the dedicated section below.

Config File Format

The config file is a CSV with four types of rules:

Text substitution - replaces literal strings anywhere in the data:

```
# Simple replacement
sensitive-hostname,single,REDACTED_HOST

# Random replacement (picks one each time)
my-secret-domain.com,random,"example1.com,example2.com,example3.com"

# Two-column shorthand (implicit single mode)
my-company.com,example.com
```

JSON field rules (`@json` prefix) - targets specific field names at any nesting depth in JSON data:

```
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER
```

Key-value tag matching - handles AWS/Azure/GCP tag structures like `{"key": "Owner", "value": "admin@corp.com"}`:

```
# Use the tag's key name (not "key" itself)
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Environment,REDACTED_ENV
```

Bulk name lists (`@names` prefix, v1.3.1+) - one config row expands at load time into one text rule per listed name, so a personal-name roster (e.g. an employee/contractor list inside a log sample) doesn't need a separate row per person. Both `single` and `random` replacement modes are supported, with the same semantics as ordinary text rules (in `random` mode each occurrence picks independently, so a given source name may map to different replacements at different points in the file):

```
# Every listed name maps to the same replacement
@names,"Alice Smith,Bob Jones,Carol Davis",single,REDACTED_NAME

# Each occurrence picks randomly from the replacement pool
@names,"Thomas Kincade,Mary Bruce,Joe Wadamaker",random,"John Doe,Jane Doe,Joe
Smith"
```

The names list is just a quoted CSV field, so it can be broken across lines for readability:

```
@names,"Thomas Kincade,
Mary Bruce,
Joe Wadamaker",random,"John Doe,Jane Doe,Joe Smith"
```

Lines starting with `#` are treated as comments.

JSON Nested Paths (v1.1)

`@json` rules may now address nested paths using dot notation. Rules without a dot continue to match any field with that name at any depth (v1.0 behavior).

```
# Unqualified - matches "accountId" anywhere in the tree (v1.0 behavior)
@json,accountId,random,"000000000001,000000000002"

# Qualified - matches "arn" ONLY when nested under userIdentity.
```

```
# Also matches "detail.userIdentity.arn" inside a CloudWatch Events envelope.
@json,userIdentity.arn,arn:aws:iam::REDACTED:user/REDACTED
```

Qualified rules match as path suffixes anchored at dotted boundaries, so `userIdentity.arn` matches `userIdentity.arn` and `detail.userIdentity.arn` but not `foo.userIdentity.arn.extra`. This is the fix for the long-standing bug where paths like `userIdentity.arn` in v1.0 configs silently did nothing.

CUI Marking Detection (CMMC / NIST SP 800-171)

CUI (Controlled Unclassified Information) is a federal data category defined by 32 CFR Part 2002 that replaced the older patchwork of FOUO/SBU/LES markings. CMMC programs are required to protect CUI per NIST SP 800-171, and logs that *contain or mention* CUI content are themselves CUI-sensitive.

Unlike PII, CUI is identified by markings, not content patterns. When the scrubber sees a CUI banner, portion marking, or legacy marking on a value or line, it replaces the entire payload with a metadata-only placeholder:

```
Input:   CUI//SP-PRVCY Employee record: John Smith DOB 01/15/1980
Output:  [CUI-REDACTED: CUI//SP-PRVCY, 57 bytes]
```

The placeholder preserves category and byte count so downstream consumers (LLMs, dashboards, normalization tools) still see the shape of the data - without any of the content.

What's detected

Banner markings (authoritative, appear at top of documents or pasted into tickets/emails):

- `CUI`, `CUI//BASIC`
- `CUI//SP-PRVCY` (Privacy), `CUI//SP-PROPIN` (Proprietary), `CUI//SP-EXPT` (Export Controlled), `CUI//SP-LEI` (Law Enforcement), `CUI//SP-CTI` (Controlled Technical Information)
- `CONTROLLED//...` (verbose form)

Portion markings (inline, precede a paragraph or field):

- `(CUI)`, `(CUI//SP-PRVCY)`, `(U//FOUO)`, `(U//SBU)`, `(U//LES)`, `(C)`, `(U)`

Legacy markings (pre-2010 but still widespread in old docs/tickets, treated as CUI-equivalent):

- `FOUO`, `SBU`, `LES`, `OUO`, `LIMDIS`, `NOFORN`, `FEDCON`, `ORCON`

CUI-adjacent content (conservative - filenames and document titles that signal CUI-regulated content):

- `ITAR`, `EAR99`, `ECCN <code>`, `DD 254`, `FCI`

Controlling CUI scrubbing

CUI detection is **enabled by default**. To disable it (e.g., for environments where this aggressive redaction is unwanted):

```
python log_scrubber.py events.csv --no-cui
```

Or in library code:

```
from log_scrubber import Scrubber
scrubber = Scrubber(config_path='log_scrubbing_config.csv', enable_cui=False)
```

Important limitations

CUI scrubbing is a *detection-then-redact* layer. If a log line contains CUI content but no marking, the scrubber has no way to know. For defense-in-depth, combine this layer with:

1. The other built-in pattern layers (PII, credentials) for content that has pattern-matchable shape.
2. Organization-specific `@json` and text rules in `log_scrubbing_config.csv` for known sensitive fields.
3. Manual review of output before any transmission. **Always spot-check samples before sending them to an internal LLM or other downstream system.**

Library Use

`log_scrubber.py` is import-safe - the CLI banner and argument parsing are guarded by `if __name__ == "__main__"`, so `import log_scrubber` from another module has no side effects.

Class API (recommended for new code):

```
from log_scrubber import Scrubber

scrubber = Scrubber(config_path='log_scrubbing_config.csv')
safe_text = scrubber.scrub(raw_log_line)

# Batch convenience
safe_batch = scrubber.scrub_many([line1, line2, line3])

# Programmatic construction with inline rules
scrubber = Scrubber(
    text_rules=[('acme.com', 'single', 'example.com')],
    json_field_rules=[('accountId', 'single', '000000000000'),
                      ('userIdentity.arn', 'single',
'arn:aws:iam::REDACTED:user/REDACTED')],
    enable_builtins=True,
    enable_cui=True,
)
```

A single `Scrubber` instance is safe to share across threads as long as the rule lists aren't mutated after construction.

Functional API (backward compatible):

```
from log_scrubber import parse_scrubbing_config, scrub_text

text_rules, json_rules = parse_scrubbing_config('log_scrubbing_config.csv')
clean = scrub_text(raw, text_rules, json_rules)

# Optional flags (default True) for v1.1 feature control
clean = scrub_text(raw, text_rules, json_rules,
                   enable_builtins=True, enable_cui=True)
```

Existing callers that don't pass the new kwargs get the v1 behavior plus the new scrubbing layers automatically.

Example Config File

```
# === Text Substitution Rules ===
ns2.com,single,example.com
sapns2,single,examplecorp
my-splunk-server,single,splunk-host

# === JSON Field Rules ===
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER
@json,sourceIPAddress,10.0.0.x
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Name,REDACTED_NAME
@json,Environment,random,"dev,staging,prod"

# === Bulk Name Lists ===
@names,"Alice Smith,Bob Jones,Carol Davis",random,"John Doe,Jane Doe,Joe Smith"
```

Command Reference

```
usage: log_scrubber.py [-h] [--config CONFIG] [--output OUTPUT]
                       [--no-builtin] [--no-cui] [--include-raw]
                       [--dry-run] [--quiet] [--version]
                       [mode] input [input ...]
```

Argument	Description
`mode`	Optional. Either `fieldsummary` or `samples`. Omit for auto-detection.
`input`	One or more input file paths. Glob patterns (`*.csv`) are expanded.
`--config`, `-c`	Path to scrubbing config CSV (auto-detected if not specified)

Argument	Description
`--output`, `-o`	Output file path (single input only; ignored in batch mode). Default: `<input>_scrubbed_<timestamp>.<ext>`
`--no-builtin`	Disable built-in regex patterns (IPs, emails, tokens, SSN, CC, etc.)
`--no-cui`	Disable CUI marking detection (enabled by default)
`--include-raw`	Include the original raw values column in fieldsummary output (default: scrubbed only)
`--dry-run`	Show what would be done without writing output
`--quiet`, `-q`	Suppress informational output
`--version`	Print version and exit

Supported Input Formats

Fieldsummary Mode

Accepts two CSV formats (auto-detected):

1. **Splunk fieldsummary export** - columns: `field`, `count`, `distinct_count`, `values`, etc.
2. **Pre-scrubbed export** - columns: `Field Name`, `Raw Values`, `Scrubbed Values`, `Count`, `Distinct Count`. This is the same format the scrubber itself writes with `--include-raw`, so re-running the tool on its own output is supported.

Both formats are handled automatically. Output column headers are normalized to `Field Name` and `Scrubbed Values` regardless of input format.

Samples Mode

Handles three formats (auto-detected):

1. **CSV with `_raw` column** - standard Splunk CSV export. The `_raw` column is scrubbed in place.
2. **Plain text** - one event per line (syslog, key=value, etc.)
3. **JSON / JSONL** - single-line or multi-line JSON events. Brace depth tracking handles multi-line JSON objects that span multiple lines.

Tips

Reuse configs across projects. The config format is plain CSV - if you already have a `log_scrubbing_config.csv` for another project or tool, copy it alongside `log_scrubber.py` and it will just work.

Review output before sending. Automated scrubbing handles known patterns, but bare personal names, free-text fields, and organization-internal terms have no distinguishing shape and slip through. See the warning in Step 5 of the workflow for the full list of common blind spots and how to address structured ones with `@json` rules - and read every row of the output before transmission.

Large JSON events (GuardDuty, CloudTrail, etc.) are handled natively - the scrubber parses JSON at any nesting depth and applies `@json` rules recursively, including AWS/Azure/GCP tag structures.

Add rules incrementally. Start with the built-in regex patterns, review the output, then add `@json` and text rules to the config for anything that slipped through.

License

Paydirt and `log_scrubber.py` are licensed under the Apache License, Version 2.0. See the `LICENSE` and `NOTICE` files in this repository for the full terms.

In short: you may use, modify, and redistribute these tools, including for commercial purposes. If you fork the project, please follow the attribution requirements in the `NOTICE` file and choose a name for your fork that doesn't include "Paydirt" or "Machine Data Insights" - those names are trademarks of Machine Data Insights Inc.

Contributing

This repository is published as-is. Bug reports and feature suggestions are welcome via email at support@machinedatainsights.com, but **we do not accept pull requests** on the public repository. This is a deliberate policy to keep the project lightweight to maintain.

If you'd like to share improvements, please feel free to fork the project and publish your version under a different name (see the `NOTICE` file), and let us know - I may incorporate similar improvements into the canonical version when appropriate.

Version History

v1.3.3

- November 8, 2026
- **Wider sample-export SPL for better TA accuracy**: the three helper queries in the *Splunk SPL for exporting samples* panel were widened so the field-value and log samples a client sends to MDI carry enough variety to develop the most accurate CIM normalization TA possible, while keeping sample bulk manageable. *Discover sourcetypes* now uses `sourcetype=*` (lists every sourcetype as a starting point; narrow it to a specific or wildcarded sourcetype as needed), *Field-value samples* raises `| fieldsummary maxvals=10` to `maxvals=15` (more distinct values per field), and *Log samples* raises `| head 20` to `| head 30` (more event variety per sourcetype). Updated in the browser tool, `log_scrubber.py`, `README.md`, and `docs/LOG_SCRUBBER_GUIDE.md`.

v1.3.2

- May 30, 2026
- **Copy to clipboard**: each result card has a new **Copy** button, between Download and Report, that copies the scrubbed output straight to the clipboard - handy for pasting a sanitized snippet into a ticket, chat, or email without saving a file first. Uses the Clipboard API with a textarea fallback for `file://` origins, and briefly confirms with a green "Copied" state. Browser-only; `log_scrubber.py` is unchanged at the CLI level (it already writes to stdout).
- **Build 2026062001** (June 20, 2026) - point-fix within v1.3.2:

    - **SPL export filename hints**: each block in the *Splunk SPL for exporting samples* panel now shows a **Save as** suggestion (`<company>_<environment>_sourcetype_discovery.csv`, `..._<sourcetype>_fieldsummary.csv`, `..._<sourcetype>_log-samples.csv`) so exports stay consistently named across environments.
    - **Wider field-value samples**: the *Field-value samples* helper now uses `| fieldsummary maxvals=10` (was `maxvals=5`) to surface more distinct values per field. Updated in the browser tool, `README.md`, and `docs/LOG_SCRUBBER_GUIDE.md`.
- **Build 2026070301** (July 3, 2026) - docs point-fix within v1.3.2 (no scrubbing behavior change):
    - **Config-editing primer**: new Editing the Scrub Config section near the top of the README covering simple/random text replacements, simple and nested `@json` replacements, and the **rule-order gotcha** (a broad rule earlier in the file can "eat" a more specific rule below it - order text rules most-specific-first).
    - **Rule-order note in the config**: the same guidance is now called out in the comments of `log_scrubbing_config.csv` (and thus the inlined default config in `Paydirt.html`), next to the `ACME Corp` / `ACME` example that demonstrates it.

v1.3.1

- May 25, 2026
- `@names` **config directive**: a single config row expands into one text rule per listed name, so bulk personal-name scrubbing (e.g. an employee/contractor roster within a log sample) no longer needs one CSV row per name. Supports both `single` and `random` replacement modes, and the names list can be broken across lines inside its quoted field for readability. See the `@names` block in `log_scrubbing_config.csv` for examples.
- **CLI and browser back in sync**: `log_scrubber.py` rejoins Paydirt.html at the same version — both engines implement `@names` with identical semantics, verified by `shared-tests/run_parity.py`.

v1.3.0

- May 17, 2026
- **Scrubbing report (audit artifact)**: each result card has a new **Report** button that downloads a retainable record of *what* was scrubbed - tool/build, timestamps, file names and sizes, **SHA-256** of the original and scrubbed text, config and rule counts, and per-category/per-rule redaction counts. Contains no source data by design; generated in-browser via SubtleCrypto. See Audit Report.
- **Audit report is browser-only in this release.** `log_scrubber.py` stays at 1.2.0, kept in sync with the Data Refinery scrubber; a CLI report counterpart is planned.

v1.2.0

- April 24, 2026
- **First public release** under the Apache License 2.0.
- **Paydirt browser tool** (`Paydirt.html`): self-contained single-file scrubber with per-field highlighting in the comparison view, summary tab with category breakdowns, and downloadable template config.
- **Parity verified** between the browser tool and the CLI by `shared-tests/run_parity.py`.
- **Demo fixture** (`paydirt_demo.log` plus companion config) covering all 25 scrubbing categories, with negative test cases.
- **Reorganized repository layout** with downloadable artifacts at the project root.
- **Build 2026051205** (May 12, 2026) - point-fixes within v1.2.0:

    - **Build numbers in the UI**: each build is now stamped with a `yyyymmddxx` build number, shown in the header next to the version pill and embedded as an HTML comment near the top of `Paydirt.html`. The same-day counter (`xx`) increments on rebuilds, so point-fixes within a version can be tracked precisely without bumping the semantic version. See `paydirt/README.md` for details.
    - **Highlight fix in the comparison view**: config search-term highlighting on the original side now uses the same literal split/join match the scrubber itself uses, instead of a regex word-boundary check. Rules like `ns2_,single,""` now correctly highlight `ns2_` inside `ns2_linux` (previously the match was silently dropped because `_` is a word character on both sides of the boundary).
    - **CSV quote round-trip**: per-cell quoting is now preserved from input to output. Cells that were explicitly quoted in the input stay quoted in the scrubbed output, eliminating cosmetic quote-stripping diffs in the side-by-side view.
- **Build 2026051702** (May 17, 2026) - point-fix within v1.2.0:
    - **Discover Sourcetypes sort order**: the *Discover sourcetypes* helper query now ends with `| sort sourcetype` instead of `| sort -event_count`, so results are ordered alphabetically by sourcetype name rather than by event volume. Updated in the browser tool, `README.md`, and `docs/LOG_SCRUBBER_GUIDE.md`.

v1.1.0

- April 20, 2026
- **CUI marking detection** (CMMC / NIST SP 800-171): banner, portion, legacy, and adjacent markings trigger full-value redaction with metadata-preserving placeholders.
- **Credential and token patterns**: AWS keys, GitHub PATs, Slack tokens, Stripe keys, JWTs, Google API keys, PEM blocks, HTTP Authorization headers, URL query-string credentials.
- **PII patterns**: SSN (valid ranges only), Luhn-validated credit cards and NPIs, formatted phone numbers, Windows user SIDs (well-known SIDs preserved).
- **Nested `@json` path support**: rules like `userIdentity.arn` match correctly; unqualified rules retain v1 behavior.
- **Auto-mode detection** and **batch processing** (multiple inputs, shell glob patterns).
- **Library API**: `Scrubber` class for importable use. Functional API remains backward compatible.
- **New flags**: `--no-cui`, `--version`.

v1.0.0

- March 13, 2026
- Initial release

Machine Data Insights Inc. *There's Gold In That Data!®* | machinedatainsights.com
Splunk is a trademark of Cisco and/or its affiliates.

Version 1.3.3 - November 8, 2026